Σε μια εναλλακτική προσέγγιση μπορούμε να αναπαραστήσουμε το τριώνυμο ως έναν πίνακα πραγματικών μήκους 3. Η συνάρτηση μπορεί να υπερφορτωθεί εύκολα ώστε να υποστηρίζεται και αυτή η αναπαράσταση του τριωνύμου, όπως δείχνει ο κώδικας 7.21.

```
static double[] triwnymo(double[] syn) { // Κώδικας 7.21
    return triwnymo(syn[0], syn[1], syn[2]);
}
```

Κώδικας 7.21 *Υπερφόρτωση της συνάρτησης triwnymo*

Σε αυτήν την περίπτωση, γίνεται η αναπαράσταση εξισώσεων β' βαθμού ευκολότερη, όπως δείχνει ο κώδικας 7.22.

```
double[] ex1 = {2, 5, -1}, ex2 = {4, 4, -9};
double[] solution1 = triwnymo(ex1);
double[] solution2 = triwnymo(ex2);
```

Κώδικας 7.22 *Αναπαράσταση εξισώσεων β βαθμού ως double[]*

7.9.3 Fibonacci

Να αναπτυχθεί συνάρτηση που επιστρέφει τον ν-οστό όρο της ακολουθίας Fibonacci.

Λύση
Καταρχάς θα πρέπει να δούμε ποιος ακριβώς είναι ο ορισμός της ακολουθίας Fibonacci. Στο διαδίκτυο μπορούμε εύκολα να τον βρούμε. Αν συμβολίσουμε τον ν-οστό με Fn, τότε ο ορισμός της ακολουθίας έχει ως εξής:

$$n = 0 \rightarrow F_n = 0$$
$$n = 1 \rightarrow F_n = 1$$
$$n > 1 \rightarrow F_n = F_{n-1} + F_{n-2}$$

Επομένως, η συνάρτησή μας θα πρέπει να λαμβάνει μία ακέραια παράμετρο, μπορεί όμως να επιστρέφει ένα long, δεδομένου πως η τιμή του ν-οστού όρου είναι σημαντικά μεγαλύτερη από το n. Ο κώδικας 7.23 δίνει μια υλοποίηση της συνάρτησης.

```
static long fibonacci(int fibIdx) { //Κώδικας 7.23
    if (fibIdx < 0) {
        throw new RuntimeException();
    } else if (fibIdx <= 1) {
        return fibIdx;
    }
    int n = 2;
    long current = 1, previous = 1, beforePrevious;
    while (++n <= fibIdx) {
        beforePrevious = previous;
        previous = current;
        current = beforePrevious + previous;
    }
    return current;
}
```

Κώδικας 7.23 *Υλοποίηση της συνάρτησης Fibonacci*

Καθώς δεν υπάρχει αρνητικός όρος της ακολουθίας, αν η συνάρτηση κληθεί με παράμετρο μικρότερη του μηδενός, σημαίνει πως κάποιο λάθος έχει συμβεί. Διαφορετικά αν η παράμετρος fibIdx είναι μικρότερη ή ίση με το 1, δηλαδή αν είναι 0 ή 1, βάσει ορισμού, επιστρέφεται η ίδια η τιμή της παραμέτρου. Αν παρόλα αυτά δεν επιστρέψει η συνάρτηση σημαίνει ότι κλήθηκε με παράμετρο μεγαλύτερη του ένα, οπότε η τιμή του ν-οστού όρου πρέπει να υπολογιστεί ως το άθροισμα των δύο προηγούμενων όρων. Η μεταβλητή n αναπαριστά

την τάξη του όρου και ξεκινά από το 2, οι δε μεταβλητές current, previous και beforePrevious αναπαριστούν τις τιμές του ν-οστού όρου, του προηγούμενού του και του όρου πριν τον προηγούμενο του ν-οστού. Ο βρόχος while επισκέπτεται έναν-έναν τους όρους της ακολουθίας έως ότου φτάσει στον όρο που εκφράζει η παράμετρος fibIdx. Σε κάθε βήμα αυτής της επαναληπτικής διαδικασίας ενημερώνονται κατάλληλα οι προηγούμενοι όροι. Με την έξοδο από την while, η current είναι ενημερωμένη με την τιμή του όρου στη θέση fibIdx.

7.9.4 Λεκτικό αριθμού

Να υλοποιηθεί συνάρτηση static String lektiko(int num) που δέχεται μια ακέραια παράμετρο από 0 έως 999 και επιστρέφει τη διατύπωσή της στα Ελληνικά.

Λύση

Σύμφωνα με την εκφώνηση και με τη διεπαφή που δίνεται μια κλήση όπως η lektiko(125) θα πρέπει να επιστρέψει την αλφαριθμητική σειρά “εκατόν είκοσι πέντε”. Εδώ έχουμε δύο θέματα να μας απασχολήσουν. Πρώτον, πόσα και ποια λεκτικά μας είναι απαραίτητα για να σχηματίσουμε την απαιτούμενη διατύπωση όλων των αριθμών από 0 έως 999 και δεύτερον πώς θα εξάγουμε πόσες εκατοντάδες έχουμε, πόσες δεκάδες και πόσες μονάδες. Προσέξτε πως αν διαιρέσετε έναν ακέραιο με το 100 (ακέραια διαίρεση), τότε λαμβάνεται τον αριθμό εκατοντάδων που περιέχει. Αν στη συνέχεια βγάλετε τις εκατοντάδες, μένει το υπόλοιπο του ακέραιου. Για παράδειγμα: Έστω num=475, h ο αριθμός των εκατοντάδων και r το υπόλοιπο, τότε h=num/100=4, r=num-h*100=75. Στη συνέχεια διαιρούμε το υπόλοιπο με το 10 για να λάβουμε τον αριθμό των δεκάδων. Τέλος, οι μονάδες είναι το υπόλοιπο μετά την αφαίρεση των δεκάδων. Ο κώδικας 7.24 επιστρέφει το λεκτικό ενός ακέραιου από το 0 έως το 999.

```
static String lektiko(int num) { //Κώδικας 7.24
    if (num < 0 || num > 999) {
        throw new RuntimeException();
    }
    if (num == 0) {
        return "μηδέν";
    }
    int ekatontades = num / 100;
    num -= ekatontades * 100;
    int dekades = num / 10;
    num -= dekades * 10;
    String hundreds = "", tens = "", units = "";
    switch (ekatontades) {
        case 0:
            break;
        case 1:
            hundreds = "εκατόν";
            break;
        case 2:
            hundreds = "διακόσια";
            break;
        case 3:
            hundreds = "τριακόσια";
            break;
        case 4:
            hundreds = "τετρακόσια";
            break;
        case 5:
            hundreds = "πεντακόσια";
            break;
        case 6:
            hundreds = "εξακόσια";
            break;
        case 7:
```

```
            hundreds = "επτακόσια";
            break;
        case 8:
            hundreds = "οκτακοσια";
            break;
        case 9:
            hundreds = "εννιακόσια";
            break;
    }
    switch (dekades) {
        case 0:
            break;
        case 1:
            tens = "δέκα";
            break;
        case 2:
            tens = "είκοσι";
            break;
        case 3:
            tens = "τριάντα";
            break;
        case 4:
            tens = "σαράντα";
            break;
        case 5:
            tens = "πενήντα";
            break;
        case 6:
            tens = "εξήντα";
            break;
        case 7:
            tens = "εβδομήντα";
            break;
        case 8:
            tens = "ογδόντα";
            break;
        case 9:
            tens = "ενενήντα";
            break;
    }
    switch (num) {
        case 0:
            break;
        case 1:
            if (tens.equals("δέκα")) {
                tens = "έντεκα";
            } else {
                units = "ένα";
            }
            break;
        case 2:
            if (tens.equals("δέκα")) {
                tens = "δώδεκα";
            } else {
                units = "δύο";
            }
            break;
        case 3:
            units = "τρία";
            break;
        case 4:
```

```
                units = "τέσσερα";
                break;
            case 5:
                units = "πέντε";
                break;
            case 6:
                units = "έξη";
                break;
            case 7:
                units = "επτά";
                break;
            case 8:
                units = "οκτώ";
                break;
            case 9:
                units = "εννέα";
                break;
        }
        String rVal = hundreds + (tens != "" ? " " : "") + tens + (units != "" ?
" " : "") + units;
        //return rVal;
        return rVal.trim();
    }
```

Κώδικας 7.24 *Μετατροπή ακέραιου σε λεκτικό*

Προσέξτε την έκφραση tens!=””?” ”:””. Σημαίνει πως αν η μεταβλητή tens έχει πάρει τιμή διάφορη από ““, τότε επιστρέφουμε ένα διάστημα που θα διαχωρίσει τις εκατοντάδες από τις δεκάδες. Αν όμως η tens παραμένει ““, τότε σημαίνει πως δεν έχουμε δεκάδες, επομένως δεν υπάρχει λόγος να προσθέσουμε το αντίστοιχο διάστημα. Ίδια λογική ισχύει και για την units!=””?” ”:””.

7.9.5 Κλάση MyArrays

Να υλοποιηθεί η κλάση MyArrays που περιλαμβάνει τις ακόλουθες στατικές συναρτήσεις χωρίς να χρησιμοποιηθεί η κλάση Arrays.

```
public static int seqSearch(int[] t, int sE)
```

Πραγματοποιεί σειριακή αναζήτηση στον t για την τιμή sE. Επιστρέφει τη θέση στον t της sE εφόσον αυτή βρεθεί, διαφορετικά -1.

```
public static int seqSearch(String[] t, String sE)
```

Πραγματοποιεί σειριακή αναζήτηση στον t για την τιμή sE. Επιστρέφει τη θέση στον t της sE εφόσον αυτή βρεθεί, διαφορετικά -1.

```
public static int cntOccurances(int[] t, int sE)
```

Μετράει και επιστρέφει το πλήθος τιμών sE στον t.

```
public static String toString(int[] t)
```

Επιστρέφει μια String αναπαράσταση του t.

```
public static String toString(int[][] t)
```

Επιστρέφει μια String αναπαράσταση του t.

```
public static int[] reverse(int[] t)
```

Επιστρέφει έναν πίνακα ακέραιων που περιέχει τα στοιχεία του t με αντεστραμμένη διάταξη.

```
public static int sum(int[] t)
```

Επιστρέφει το άθροισμα των στοιχείων του t.

```
public static int max(int[] t)
```

Επιστρέφει τη θέση του μέγιστου στοιχείου του t.

```
public static boolean equals(int[] tA, int[] tB)
```

Επιστρέφει true αν κάθε στοιχείο του tA είναι και στοιχείο του tB και κάθε στοιχείο του tB είναι και στοιχείο του tA.

```
public static boolean equals(String[] tA, String[] tB)
```

Επιστρέφει true αν κάθε στοιχείο του tA είναι και στοιχείο του tB και κάθε στοιχείο του tB είναι και στοιχείο του tA.

```
public static int binarySearch(int[] t, int sE)
```

Πραγματοποιεί δυαδική αναζήτηση στον t για την τιμή sE. Αν η τιμή βρεθεί, επιστρέφει τη θέση της στον t, διαφορετικά επιστρέφει -1.

```
public static void swap(int[] t, int idx1, int idx2)
```

Αντιμεταθέτει το στοιχείο στη θέση idx1 του t με το στοιχείο στη θέση idx2.

```
public static int idxOfMin(int[] array, int startIdx)
```

Επιστρέφει τη θέση του ελάχιστου στοιχείου του t από τη θέση startIdx και μετά.

```
public static void sort(int[] t)
```

Ταξινομεί κατ'αύξουσα αριθμητική σειρά των πίνακα t.

Λύση
Ο κώδικας 7.25 παραθέτει μια λύση της άσκησης και ακολουθούν σχόλια.

```
public class MyArrays {

    public static int seqSearch(int[] t, int sE) {
        for (int i = 0; i < t.length; i++) {
            if (t[i] == sE) {
                return i;
            }
        }
        return -1;
    }

    public static int seqSearch(String[] t, String sE) {
        for (int i = 0; i < t.length; i++) {
            if (t[i].equals(sE)) {
```

```
                return i;
            }
        }
        return -1;
    }

    public static int cntOccurances(int[] t, int sE) {
        int cnt = 0;
        for (int e : t) {
            if (e == sE) {
                cnt++;
            }
        }
        return cnt;
    }

    public static String toString(int[] t) {
        if (t.length == 0) {
            return "[]";
        }
        String s = "[";
        for (int i = 0; i < t.length; i++) {
            s += t[i];
            if (i < t.length - 1) {
                s += ", ";
            }
        }
        s += "]";
        return s;
    }

    public static String toString(int[][] t) {
        if (t.length == 0) {
            return "[]";
        }
        String s = "[";
        for (int i = 0; i < t.length; i++) {
            s += toString(t[i]);
            if (i < t.length - 1) {
                s += ",\n";
            }
        }
        s += "]";
        return s;
    }

    public static int[] reverse(int[] input) {
        int[] rVal = new int[input.length];
        for (int i = 0; i < input.length; i++) {
            rVal[rVal.length - 1 - i] = input[i];
        }
        return rVal;
    }

    public static int sum(int[] t) {
        int sum = 0;
        for (int i : t) {
            sum += i;
        }
        return sum;
    }
```

```
public static int max(int[] t) {
    int max = t[0];
    int idx = 0;
    for (int i = 1; i < t.length; i++) {
        if (t[i] > max) {
            idx = i;
            max = t[i];
        }
    }
    return idx;
}

public static boolean equals(int[] tA, int[] tB) {
    if (tA.length != tB.length) {
        return false;
    }
    for (int cE : tA) {
        if (seqSearch(tB, cE) == -1) {
            return false;
        }
    }
    return true;
}

public static boolean equals(String[] tA, String[] tB) {
    if (tA.length != tB.length) {
        return false;
    }
    for (String cE : tA) {
        if (seqSearch(tB, cE) == -1) {
            return false;
        }
    }
    return true;
}

public static int binarySearch(int[] tbl, int schElement) {
    int from = 0, to = tbl.length - 1, mid = (to - from) / 2;
    while (to >= from) {
        if (tbl[mid] == schElement) {
            return mid;
        } else if (schElement < tbl[mid]) {
            to = mid - 1;
        } else {
            from = mid + 1;
        }
        mid = from + (to - from) / 2;
    }
    return -1;
}

public static void swap(int[] t, int idx1, int idx2) {
    int i = t[idx1];
    t[idx1] = t[idx2];
    t[idx2] = i;
}

public static int idxOfMin(int[] t, int startIdx) {
    int min = t[startIdx];
    int rVal = startIdx;
```

```
        for (int i = startIdx + 1; i < t.length; i++) {
            if (t[i] < min) {
                min = t[i];
                rVal = i;
            }
        }
        return rVal;
    }

    public static void sort(int[] t) {
        for (int i = 0; i < t.length; i++) {
            int mIdx = idxOfMin(t, i);
            if (mIdx != i) {
                swap(t, mIdx, i);
            }
        }
    }

}
```

Κώδικας 7.25 *Η κλάση MyArrays*

Οι δύο πρώτες συναρτήσεις είναι απλές. Η πρώτη κάνει σειριακή αναζήτηση σε πίνακα ακέραιων και η δεύτερη σε πίνακα αλφαριθμητικών σειρών. Η μόνη λεπτομέρεια που πρέπει να προσέξουμε είναι πως για μεν τους ακέραιους, η σύγκριση γίνεται με τον τελεστή ==, για δε τις αλφαριθμητικές σειρές, η σύγκριση γίνεται με τη συνάρτηση equals.

Η cntOccurances επίσης βασίζεται σε σειριακή προσπέλαση όλων των στοιχείων του πίνακα. Κάθε φορά που βρίσκει τιμή ίση με τη δεύτερη παράμετρό της, αυξάνει έναν μετρητή τον οποίον και επιστρέφει.

Η toString για μονοδιάστατο πίνακα ακέραιων, καταρχάς επιστρέφει [] αν η παράμετρός της έχει μήκος μηδέν. Στη συνέχεια, αρχικοποιεί την τοπική μεταβλητή s στη σειρά [. Μετά προσπελαύνει ένα προς ένα τα στοιχεία τα στοιχεία του t και προσθέτει στο s την τιμή του στοιχείου ακολουθούμενη από κόμμα και διάστημα. Εξαιρεί όμως την πρόσθεση κόμματος και διαστήματος στο τελευταίο στοιχείο. Τέλος, κλείνει το s προσθέτοντας το].

Η toString για δισδιάστατο πίνακα έχει ανάλογη λογική με την toString για μονοδιάστατο. Μόνο που εδώ τα στοιχεία είναι πίνακες, οπότε χρησιμοποιεί την toString για να πάρει την αναπαράσταση των μονοδιάστατων πινάκων στοιχείων του t.

Η reverse ορίζει τον πίνακα rVal με μήκος ίσο με το μήκος του πίνακα input. Στη συνέχεια αντιγράφει ένα προς ένα τα στοιχεία του input στον rVal σε αντιδιαμετρικά αντίθετη θέση. Προσέξτε πως όταν ο δείκτης i είναι ίσος με μηδέν, δηλαδή αναφερόμαστε στην πρώτη θέση του input, η έκφραση rVal.length-1-i αναφέρεται στην τελευταία θέση του rVal, όταν ο δείκτης i είναι ίσος με 1, δηλαδή αναφερόμαστε στη δεύτερη θέση του input, η έκφραση rVal.length-1-i αναφέρεται στην προτελευταία θέση του rVal, κοκ.

Η sum εφαρμόζει σειριακή προσπέλαση και προσθέτει κάθε τιμή του t στην τοπική μεταβλητή sum την οποία τελικά επιστρέφει.

Παρόμοια η max εφαρμόζει σειριακή προσπέλαση. Αρχικοποιεί την τοπική μεταβλητή max στην τιμή στη θέση μηδέν του t. Στη συνέχεια, επισκέπτεται τα στοιχεία του t από τη θέση 1 και μετά και κάθε φορά που βρίσκει μεγαλύτερη τιμή ενημερώνει την max και την idx που κρατάει τη θέση της max. Επιστρέφει την idx.

Η equals για πίνακες ακέραιων συγκρίνει καταρχάς τα μήκη των πινάκων. Αν δεν είναι ίσα, επιστρέφει false. Αν δεν επιστρέψει από τον έλεγχο των μηκών, ψάχνει στη συνέχεια αν κάθε στοιχείο του tA είναι και στοιχείο του tB. Προσέξτε πως ο αντίστροφος έλεγχος είναι περιττός από τη στιγμή που τα μήκη είναι ίσα.

Ίδια είναι και η λογική της equals για πίνακες από Strings.

Η binarySearch εφαρμόζει τη λογική που αναπτύξαμε στην ενότητα 6.5.2.

Η swap κρατάει την πρώτη από τις τιμές που αντιμεταθέτει σε προσωρινή μεταβλητή, εκχωρεί τη δεύτερη τιμή στην πρώτη και στη θέση της δεύτερης τιμής εκχωρεί την τιμή της προσωρινής μεταβλητής.

Η sort υλοποιεί τον αλγόριθμο που παρατίθεται στον κώδικα 6.18. Η διαφορά της είναι πως την εύρεση του μικρότερου στοιχείου την αναθέτει στην idxMin.

7.9.6 Ταξινόμηση δισδιάστατου πίνακα με βάση το άθροισμα γραμμών

Να υλοποιηθεί συνάρτηση με διεπαφή public static void sortSubArraysBySum(int[][] t) που ταξινομεί τους πίνακες του t κατά αύξουσα αριθμητική σειρά του αθροίσματος των στοιχείων τους. Για παράδειγμα, ο πίνακας

```
{
  {100, 200, 300, 400},
  {1, 2, 3, 4, 5},
  {10, 20, 30, 40, 50, 60}
}
```

θα πρέπει να ταξινομηθεί ως

```
{
   {1, 2, 3, 4, 5},
   {10, 20, 30, 40, 50, 60},
   {100, 200, 300, 400}
}
```

καθώς 1+2=3+4+5 < 10+20+30+40+50+60 < 100+200+300+400΄

Λύση
Θα εφαρμόσουμε παρόμοια λογική με αυτήν της ταξινόμησης μονοδιάστατου πίνακα. Έτσι, θα χρειαστούμε μια συνάρτηση που εντοπίζει τον πίνακα-στοιχείο του t που έχει το μικρότερο άθροισμα.

```
static int idxOfMinSum(int[][] t, int startIdx) {
    int min = MyArrays.sum(t[startIdx]);
    int rVal = startIdx;
    for (int i = startIdx + 1; i < t.length; i++) {
        if (MyArrays.sum(t[i]) < min) {
            min = MyArrays.sum(t[i]);
            rVal = i;
        }
    }
    return rVal;
}
```

Κώδικας 7.26 *Εύρεση πίνακα στοιχείου με το μικρότερο άθροισμα*

Ο κώδικας 7.26 παρουσιάζει την idxOfMinSum που αρχίζοντας μετά από τη θέση startIdx εντοπίζει ποιος πίνακας-στοιχείο του t έχει το μικρότερο άθροισμα στοιχείων.
Έχοντας στη διάθεσή μας την idxOfMinSum, μπορούμε να υλοποιήσουμε την απαιτούμενη ταξινόμηση εφαρμόζοντας αντίστοιχη λογική με αυτήν της ταξινόμησης μονοδιάστατου πίνακα. Ο κώδικας 7.27 παρουσιάζει τη σχετική υλοποίηση:

```
public static void sortSubArraysBySum(int[][] t) { //Κώδικας 7.27
    for (int i = 0; i < t.length; i++) {
        int mIdx = idxOfMinSum(t, i);
        if (mIdx != i) {
            int[] local = t[i];
            t[i] = t[mIdx];
            t[mIdx] = local;
        }
    }
}
```

Κώδικας 7.27 *Ταξινόμηση σύμφωνα με το άθροισμα των γραμμών δισδιάστατου πίνακα*

7.9.7 Διαχείριση ψηφιοσειρών

Να υλοποιηθεί η κλάση BitMaps που περιλαμβάνει τις λειτουργίες

```
public static int setBit(int i, int p)
```
Θέτει στο bit που βρίσκεται στη θέση p αντιγράφου του ακέραιου i την τιμή 1. Επιστρέφει τον παραγόμενο ακέραιο.

```
public static int resetBit(int i, int p)
```
Θέτει στο bit που βρίσκεται στη θέση p αντιγράφου του ακέραιου i την τιμή 0. Επιστρέφει τον παραγόμενο ακέραιο.

```
public static int getBit(int i, int p)
```
Επιστρέφει την τιμή του bit που βρίσκεται στη θέση p του ακέραιου i

```
public static int reverseBit(int i, int p)
```
Αντιστρέφει σε αντίγραφο του i το bit στη θέση p. Επιστρέφει τον παραγόμενο ακέραιο.

Λύση
Παρουσιάζουμε τον κώδικα της κλάσης BitMap ακολουθούμενο από τις αναγκαίες επεξηγήσεις.

```
public class BitMap { //Κώδικας 7.28

    public static int setBit(int i, int p) {
        int local = 1;
        local <<= p;
        return i | local;
    }

    public static int resetBit(int i, int p) {
        int local = setBit(0, p);
        local = ~local;
        return i & local;
    }

    public static int getBit(int i, int p) {
        int local = setBit(0, p);
        if ((local & i) == local) {
            return 1;
        }
        return 0;
    }

    public static int reverseBit(int i, int p) {
        if (getBit(i, p) == 1) {
            return resetBit(i, p);
        } else {
            return setBit(i, p);
        }
    }

    public static void main(String[] args) {
        System.out.println(setBit(1, 1));
        System.out.println(resetBit(3, 0));
        System.out.println(getBit(3, 1));
        System.out.println(getBit(3, 2));
        System.out.println(reverseBit(3,0));
    }
}
```

Κώδικας 7.28 *Διαχείριση ψηφιοσειρών*

Η συνάρτηση setBit, στην πρώτη γραμμή αρχικοποιεί τη μεταβλητή local στην τιμή 1. Επομένως, ο ψηφιοχάρτης της local γίνεται 00000000000000000000000000000001, δηλαδή 31 μηδενικά ακολουθούμενα από έναν άσο. Παρότι ο int έχει μήκος 32 bits, για απλοποίηση, στο εξής θα θεωρούμε ότι είναι μήκους 8 bits. Με αυτήν την υπόθεση, η local έχει το ακόλουθο περιεχόμενο 00000001. Στη συνέχεια εκτελείται διολίσθηση στα αριστερά για όσες θέσεις καθορίζει η παράμετρος p. Για παράδειγμα, θεωρήστε την κλήση setBit(1,1) στην πρώτη γραμμή της main. Η p εδώ έχει την τιμή 1. Επομένως, η local θα αποκτήσει περιεχόμενο 00000010. Στη συνέχεια γίνεται bitwise OR μεταξύ της παραμέτρου i και της local. Η παράμετρος i έχει τιμή 1, δηλαδή περιεχόμενο 00000001. Συνεπώς 00000010 | 00000001=00000011, δηλαδή 3 στο δεκαδικό σύστημα που είναι και η τιμή που εμφανίζεται από την println στην πρώτη γραμμή της main.

Με άλλα λόγια διαμορφώσαμε έναν ψηφιοχάρτη στη μεταβλητή local ο οποίος έχει όλα τα bits ίσα με το 0, εκτός από το bit που θέλουμε να κάνουμε 1. Στη συνέχεια, το bitwise OR μεταξύ της local και οποιουδήποτε ακέραιου, x, θα μετατρέψει σε 1 το bit του x που βρίσκεται στη θέση που έχει η local τιμή 1.

Παρόμοια εργαζόμαστε και για την resetBit. Ορίζουμε τη μεταβλητή local που έχει όλα τα bits 0 εκτός από το bit στη θέση p. Με bitwise not αντιστρέφουμε όλα τα bits της local οπότε σε όλες τις θέσεις έχει πλέον 1 εκτός από τη θέση p που έχει 0. Στη συνέχεια το bitwise and με την παράμετρο i, έχει ως αποτέλεσμα όλα τα bits να παραμείνουν ως έχουν εκτός από το bit στη θέση p που αποκτά την τιμή 0.

Η getBit δημιουργεί τον ακέραιο local που αρχικοποιείται ώστε ο ψηφιοχάρτης του να αποτελείται από 0 εκτός από τη θέση p. Επομένως, αν το bitwise and μεταξύ της local και της i επιστρέψει ακέραιο ίσο με την local αυτό σημαίνει πως το bit στη θέση p της i είναι 1 αλλιώς 0.

Τέλος η reverseBit είναι απλή καθώς βασίζεται στις getBit, resetBit και setBit και δεν κάνει απευθείας πράξεις σε επίπεδο ψηφίου.

7.10 Ασκήσεις προς λύση

1. Να αναπτύξετε συνάρτηση double abs(double n) που επιστρέφει την απόλυτη τιμή του n.

2. Να αναπτύξετε συνάρτηση που υπολογίζει και τυπώνει την τιμή της $f(x)=x^4+5x^3+2x^2+8x+1$ για τις τιμές του x=1..100, όπου x ακέραιος.

3. Να αναπτύξετε συνάρτηση που τυπώνει τους ν πρώτους όρους της ακολουθίας Fibonacci.

4. Να αναπτυχθεί συνάρτηση που επιστρέφει true αν η ακέραιη παράμετρός της είναι πρώτος αριθμός.

5. Να αναπτυχθεί συνάρτηση που τυπώνει τους πρώτους αριθμούς από το 1 έως το 100.

6. Να αναπτυχθεί συνάρτηση που τυπώνει τους αριθμούς από το 1 έως το 100 που δεν είναι πρώτοι.

7. Να αναπτυχθεί συνάρτηση που λαμβάνει μια boolean παράμετρο και επιστρέφει έναν πίνακα με τους χαρακτήρες της Αγγλικής, κεφαλαίους ή πεζούς, ανάλογα με την τιμή της παραμέτρου.

8. Να υλοποιηθεί Java application που λαμβάνει 3 παραμέτρους που αντιπροσωπεύουν μια εξίσωση 2ου βαθμού ($ax^2+bx+c=0$) και τυπώνει τη λύση της εφόσον υπάρχει ή διαφορετικά το μήνυμα δεν υπάρχει λύση, διακρίνουσα <0.

9. Να αναπτυχθεί συνάρτηση που επιστρέφει το άθροισμα των ψηφίων ενός ακεραίου.

10. Να αναπτυχθεί συνάρτηση με διεπαφή static int max(int…in) που λαμβάνει μια σειρά από ακέραιους και επιστρέφει τον μεγαλύτερο.

11. Να αναπτυχθεί συνάρτηση που λαμβάνει ως παράμετρο έναν πραγματικό αριθμό που αναπαριστά θερμοκρασία σε Celsius και επιστρέφει τη θερμοκρασία σε Fahrenheit.

12. Υπερφορτώστε την equals της άσκησης 7.8.5 ώστε να εξετάζει και πίνακες πραγματικών.

13. Αναπτύξτε συνάρτηση με διεπαφή static int[] resize(int[] t, int newSize) που επιστρέφει ένα αντίγραφο του t με μήκος newSize. Αν η τιμή της newSize είναι μικρότερη από το αρχικό μήκος του t, τότε ενδέχεται να απωλεσθούν δεδομένα από τον πίνακα.

14. Οποιοσδήποτε βαθμός μικρότερος του 5 αντιστοιχεί σε αποτυχία. Η βαθμολογία από 5 έως 6,4 αντιστοιχεί στον χαρακτηρισμό «καλώς», από 6,5 έως 8,4 στο «λίαν καλώς» και από 8,5 έως 10 στο «άριστα». Να

αναπτύξετε συνάρτηση που λαμβάνει ως είσοδο έναν βαθμό από 0 έως 10 και επιστρέφει τον αντίστοιχο χαρακτηρισμό.

15. Να αναπτυχθεί εφαρμογή Java που παράγει 100 τυχαίους αριθμούς από το 1 έως το 40 και στη συνέχεια μετράει.

- Πόσοι από τους παραγόμενους αριθμούς είναι μοναδικοί.
- Πόσοι είναι άρτιοι και πόσοι περιττοί.
- Ποιοι αριθμοί από το 1 έως το 40 δεν υπάρχουν καθόλου στην παραγόμενη ακολουθία.
- Ποιος ή ποιοι αριθμοί εμφανίζονται περισσότερες φορές.

20. Να αναπτυχθεί συνάρτηση με διεπαφή static void reverse(char[] t) που αντιστρέφει τις θέσεις των στοιχείων του t, δηλαδή τοποθετεί το στοιχείο από την πρώτη στην τελευταία θέση του πίνακα, από τη δεύτερη στην προτελευταία, κοκ.

21. Να αναπτυχθεί συνάρτηση που επιστρέφει τον μέγιστο κοινό διαιρέτη ενός πλήθους ακέραιων.

22. Να αναπτυχθεί συνάρτηση που επιστρέφει το ελάχιστο κοινό πολλαπλάσιο ενός πλήθους ακέραιων.

23. Να αναπτύξετε συνάρτηση που λαμβάνει ως παράμετρο έναν πίνακα ακέραιων και επιστρέφει επίσης έναν πίνακα ακέραιων που αναπαριστά το σύνολο των ακέραιων του πίνακα εισόδου. Προσέξτε πως σε ένα σύνολο, ένα στοιχείο μπορεί να υπάρχει μόνο μία φορά.

24. Να αναπτύξετε συνάρτηση που υλοποιεί την ταξινόμηση φυσαλίδας σε πίνακα πραγματικών.

25. Να αναπτύξετε συνάρτηση που εξετάζει αν ένας πίνακας είναι ταξινομημένος. Η συνάρτηση να επιστρέφει 0 για αταξινόμητους πίνακες, -1 για ταξινομημένους κατά φθίνουσα σειρά και 1 για ταξινομημένους κατά αύξουσα σειρά.

26. Να αναπτύξετε συνάρτηση που συγχωνεύει δύο ταξινομημένους πίνακες σε έναν νέο πίνακα επίσης ταξινομημένο.

27. Να αναπτύξετε συνάρτηση που λαμβάνει ως παράμετρο δύο μονοδιάστατους πίνακες ακέραιων και επιστρέφει έναν πίνακα που περιέχει μόνο τα κοινά στοιχεία των παραμέτρων της έτσι ώστε κάθε κοινό στοιχείο να υπάρχει μόνο μία φορά στον πίνακα επιστροφής.

28. Να αναπτύξετε συνάρτηση που λαμβάνει ως παράμετρο δύο τετραγωνικούς πίνακες πραγματικών και επιστρέφει το άθροισμά τους. Σημειώστε πως τετραγωνικοί λέγονται οι πίνακες που έχουν ίδιο αριθμό γραμμών και στηλών.

29. Να αναπτύξετε συνάρτηση που ανακατανέμει τα στοιχεία ενός μονοδιάστατου πίνακα.

30. Να αναπτύξετε συνάρτηση που ανακατανέμει τα στοιχεία ενός δισδιάστατου πίνακα έτσι ώστε ένα τελικό στοιχείο να είναι πιθανό να βρεθεί σε οποιαδήποτε γραμμή και στήλη του πίνακα.

31. Ο τόκος ενός κεφαλαίου υπολογίζεται με βάση το επιτόκιο. Όταν ο τόκος μίας περιόδου προστίθεται στο κεφάλαιο, τότε ο τόκος της επόμενης περιόδου υπολογίζεται επί του αρχικού κεφαλαίου αυξημένου κατά τον τόκο της προηγούμενης περιόδου. Το φαινόμενο αυτό ονομάζεται ανατοκισμός. Να αναπτύξετε συνάρτηση που λαμβάνει ως παραμέτρους το κεφάλαιο, το επιτόκιο και τον αριθμό των περιόδων και υπολογίζει το σύνολο του τόκου.

32. Να αναπτυχθεί συνάρτηση που επιστρέφει την κύρια διαγώνιο τετραγωνικού πίνακα.

33. Να αναπτυχθεί συνάρτηση που επιστρέφει το Ελάχιστο Κοινό Πολλαπλάσιο μιας σειράς ακέραιων.

34. Να αναπτυχθεί συνάρτηση που επιστρέφει τον Μέγιστο Κοινό Διαιρέτη μιας σειράς ακέραιων.

35. Έστω nb ένας αριθμός εκφρασμένος σε θεσιακό αριθμητικό σύστημα με βάση b. Η μετατροπή του στο δεκαδικό σύστημα αρίθμησης γίνεται από τον τύπο

$$n_b = s_k \times b^k + s_{k-1} \times b^{k-1} + \cdots + s_1 \times b^1 + s_0 \times b^0 + s_{-1} \times b^{-1} + s_{-2} \times b^{-2} + \cdots + s_m \times b^m$$